

MANUAL TÁCTICO DE COMPETENCIA ICPC

Equipo TRD — Guía Avanzada de Reconocimiento, Índices y Modificabilidad de Plantillas

1. Estrategia y Dinámica de Equipo

1.1 Protocolo de la Máquina Única

En una competencia ICPC con 1 sola computadora para 3 personas, el recurso más escaso es el tiempo de máquina. El cuello de botella casi nunca es la velocidad de tipeo, sino el tiempo desperdiciado escribiendo código no probado en papel.

- **Regla del Papel:** Nadie se sienta a programar sin tener escritas las transiciones del DP, la condición de parada o el diseño de la red de flujo con sus capacidades.
- **Protocolo de 15 minutos:** Si estás en la máquina y llevas 15 minutos atascado en un bug, **imprime el código y cede la computadora**. Depura con lápiz sobre el papel mientras tu compañero avanza en otro problema ya diseñado.
- **Roles y Fuertes del Equipo TRD:**
 - **JuanJoe-18:** Sorting, Greedy, Geometría Computacional, Grafos, Range Queries (RQ) y Strings. Identificación ágil de estructuras y reducciones de búsqueda.
 - **“Botinix”:** Programador COMPLETO e implementador principal. Tracción pesada, consistencia, velocidad en templates y resolución de bugs difíciles.
 - **JuanchoKing:** Rey del Chess, Matemáticas puras, Teoría de Números, Combinatoria y sólido implementador. Maestro del juego y de las reducciones abstractas.

1.2 Template Base y Manejo de I/O

Nunca uses `endl`; usa `'\n'`. Asegúrate de desactivar la sincronización con `stdio`:

```
ios_base::sync_with_stdio(false);
cin.tie(NULL);
```

Para números aleatorios seguros (evitando hacks de hashes con pruebas premeditadas en Codeforces/ICPC):

```
mt19937_64 rng(chrono::steady_clock::now().
    time_since_epoch().count());
ll rand_val(ll a, ll b) {
    return uniform_int_distribution<ll>(a, b)(rng);
}
```

2. Sistema Canónico de Índices (0 vs 1)

Uno de los bugs más devastadores en vivo es mezclar 0-based y 1-based. Esta tabla resume la convención exacta de nuestra biblioteca:

Estructura	Archivo	Base	Límites
Graph<T>	graph.hpp	1-based	$1 \dots n$
Dijkstra<T>	graph.hpp	1-based	$1 \dots n$
ZeroOneBfs	graph.hpp	1-based	$1 \dots n$
DSU	graph.hpp	0-based	$0 \dots n - 1$
FenwickTree	rq.hpp	1-based	$1 \dots n$
SparseTable	rq.hpp	0-based	$0 \dots n - 1$
IterSegTree	rq.hpp	0-based	$0 \dots n - 1$
HLD	tree.hpp	1-based	$1 \dots n$
LCA / BinLift	tree.hpp	1-based	$1 \dots n$

2.1 El Peligro Fatal de Fenwick (BIT)

En el Fenwick Tree:

```
void add(int idx, ll val) {
    for (; idx <= n; idx += idx & -idx) bit[idx] += val;
}
```

Si se llama con `idx = 0`, la operación `0 & -0` da `0`, resultando en un **bucle infinito** que genera TLE inexplicable. Si tus datos son 0-based, haz siempre `ft.add(x + 1, val)` y `ft.query(1 + 1, r + 1)`.

2.2 DSU con Problemas 1-based

Como el constructor de `DSU(n)` inicializa de `0` a `n - 1`, si el problema da vértices $1 \dots n$, simplemente inicializa `dsu(n + 1)`. El nodo `0` existirá pero quedará aislado sin afectar las componentes $1 \dots n$.

2.3 Euler Tour (Aplanamiento de Árbol)

Para mapear subárboles a rangos contiguos en un Segment Tree:

```
int timer = 0;
vi tin(n + 1), tout(n + 1), flat(n + 1);
void dfs_flat(int u, int p) {
    tin[u] = ++timer;
    flat[timer] = u;
    for (int v : adj[u]) if (v != p) dfs_flat(v, u);
    tout[u] = timer;
}
// El subarbol de u es exactamente el rango: [tin[u], tout[u]]
```

3. Modificabilidad Avanzada: Recetario de Funciones

3.1 Dijkstra Multi-Estado (Estado Extendido)

Cuando el costo no solo depende del vértice, sino de un recurso (ej. K boletos de descuento, paridad de pasos, combustible):

1. Define el estado como una tupla: **(nodo, recurso)**.
2. La matriz de distancias pasa a ser `dist[N][K+1]`.

3. El heap contiene {costo, {nodo, recurso}}.

```
vector<vll> dist(n + 1, vll(k + 1, LINF));
using Item = pair<ll, pair<int, int>>;
priority_queue<Item, vector<Item>, greater<Item>> pq;

dist[src][k] = 0;
pq.push({0, {src, k}});

while (!pq.empty()) {
    auto [d, state] = pq.top(); pq.pop();
    auto [u, rem] = state;
    if (d > dist[u][rem]) continue;

    for (int id : g.adj[u]) {
        auto& e = g.edges[id];
        // Transición normal (costo completo)
        if (dist[u][rem] + e.weight < dist[e.to][rem]) {
            dist[e.to][rem] = dist[u][rem] + e.weight;
            pq.push({dist[e.to][rem], {e.to, rem}});
        }
        // Transición especial (usando 1 descuento)
        if (rem > 0) {
            ll w2 = dist[u][rem] + e.weight / 2;
            if (w2 < dist[e.to][rem - 1]) {
                dist[e.to][rem - 1] = w2;
                pq.push({w2, {e.to, rem - 1}});
            }
        }
    }
}
```

3.2 Reconstrucción de Caminos (Path Recovery)

Para recuperar la ruta en Dijkstra, BFS o 0-1 BFS:

```
// Vector de padres inicializado en -1: vi p(n + 1, -1);
// Al relajar: p[e.to] = u;
vi restore_path(int target, const vi& p) {
    vi path;
    for (int v = target; v != -1; v = p[v]) path.pb(v);
    reverse(all(path));
    return path; // Retorna {src, ..., target}
}
```

3.3 Segment Tree: Desacoplamiento de Nodo

No modifiques los índices del árbol. Solo redefine el **struct Node** y la función **merge**:

```
// Ejemplo: Max Subarray Sum en  $O(\log N)$ 
struct Node {
    ll sum, pref, suff, ans;
    Node(ll v = 0) {
        sum = v;
        pref = suff = ans = max(0LL, v);
    }
};

Node merge(const Node& a, const Node& b) {
    Node res;
    res.sum = a.sum + b.sum;
    res.pref = max(a.pref, a.sum + b.pref);
    res.suff = max(b.suff, b.sum + a.suff);
    res.ans = max({a.ans, b.ans, a.suff + b.pref});
    return res;
}
```

3.4 Segment Tree con Lazy Propagation Canónico

El secreto para no confundirse con Lazy es respetar dos reglas:

- **Aplicar al nodo:** Al aplicar una etiqueta a un nodo, se debe actualizar su valor real y combinar su etiqueta lazy pendiente.
- **Empujar a los hijos:** Solo empuja a los hijos antes de descender.

```
struct LazySegTree {
    int n;
    vll st, lazy;
    LazySegTree(int n) : n(n), st(4*n, 0), lazy(4*n, 0) {}

    void apply(int p, int l, int r, ll val) {
        st[p] += (r - l + 1) * val; // Actualiza valor
        lazy[p] += val;           // Acumula tag
    }

    void push(int p, int l, int r) {
        if (lazy[p] == 0) return;
        int mid = (l + r) / 2;
        apply(2*p, l, mid, lazy[p]);
        apply(2*p+1, mid+1, r, lazy[p]);
        lazy[p] = 0;
    }

    void update(int p, int l, int r, int ql, int qr, ll val) {
        if (ql <= l && r <= qr) { apply(p, l, r, val); return; }
        push(p, l, r);
        int mid = (l + r) / 2;
        if (ql <= mid) update(2*p, l, mid, ql, qr, val);
        if (qr > mid) update(2*p+1, mid+1, r, ql, qr, val);
        st[p] = st[2*p] + st[2*p+1];
    }

    ll query(int p, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return st[p];
        push(p, l, r);
        int mid = (l + r) / 2;
        ll res = 0;
        if (ql <= mid) res += query(2*p, l, mid, ql, qr);
        if (qr > mid) res += query(2*p+1, mid+1, r, ql, qr);
        return res;
    }
};
```

3.5 HLD sobre Aristas (Edge Updates)

Para actualizar o consultar caminos con valores en las aristas:

1. Cada arista (u, v) se asigna al nodo hijo: $\text{child} = (\text{depth}[u] > \text{depth}[v] ? u : v)$.
2. Al recorrer cadenas en HLD, cuando ambos nodos u y v quedan en la misma cadena pesada con $\text{pos}[u] \leq \text{pos}[v]$, la consulta debe ser:

```
if (u != v) {
    res = merge(res, segtree.query(pos[u] + 1, pos[v]));
}
```

El **+1** excluye la arista que sube desde el LCA hacia su padre.

3.6 DP con Rerooting (Todos los Nodos como Raíz)

Problema tipo: "Calcula para cada nodo u la suma de distancias / mejor camino si u fuera la raíz del árbol".

- **Paso 1 (DFS Bottom-up):** Calcula $dp[u]$ combinando los estados de sus hijos.

- **Paso 2 (DFS Top-down / Rerooting):** Al pasar la raíz de u a un hijo v , se extrae la contribución de v del estado de u (usando prefijos y sufijos de hijos si la operación no es invertible) y se le añade a v como nuevo hijo.

```
struct State { ll sz, sum; };
State merge_sub(State a, State b) {
    return {a.sz + b.sz, a.sum + b.sum + b.sz};
}
// Al mover la raíz de u a v:
// 1. Quitar a v de u
// 2. Insertar u en v
```

3.7 Optimización Divide and Conquer (D&C DP)

Aplica a recurrencias de la forma:

$$dp[i][j] = \min_{k < j} (dp[i-1][k] + C[k][j])$$

donde el punto óptimo $opt[i][j]$ es monótono: $opt[i][j-1] \leq opt[i][j] \leq opt[i][j+1]$. Reduce la complejidad de $O(KN^2)$ a $O(KN \log N)$.

```
void compute(int i, int l, int r, int optl, int optr) {
    if (l > r) return;
    int mid = (l + r) / 2;
    pair<ll, int> best = {LLINF, -1};
    for (int k = optl; k <= min(mid - 1, optr); k++) {
        best = min(best, {dp[i-1][k] + cost(k, mid), k});
    }
    dp[i][mid] = best.first;
    int opt = best.second;
    compute(i, l, mid - 1, optl, opt);
    compute(i, mid + 1, r, opt, optr);
}
```

3.8 SOS DP (Sum Over Subsets)

Dada una función A sobre máscaras de bits, calcula para cada máscara $mask$ la suma de todos sus subconjuntos $sub \subseteq mask$:

$$F[mask] = \sum_{sub \subseteq mask} A[sub]$$

Fuerza bruta sobre submáscaras toma $O(3^N)$. Con SOS DP se logra en $O(N2^N)$:

```
// dp inicializado con los valores base a[mask]
for (int i = 0; i < N; i++) {
    for (int mask = 0; mask < (1 << N); mask++) {
        if (mask & (1 << i)) {
            dp[mask] += dp[mask ^ (1 << i)];
        }
    }
}
```

4. Marco Teórico: DP como Autómatas y DAGs

Cuando un problema de Programación Dinámica parece inabordable o multidimensional (DP “HARD”), modelarlo formalmente como un **Autómata Finito Determinista (DFA)** y su **Grafo Acíclico Dirigido (DAG)** inducido elimina toda ambigüedad.

4.1 La 5-Tupla Formal $\mathcal{M} = (Q, \Sigma, \delta, q_0, F)$

Todo problema de DP se formaliza rigurosamente como:

- **Espacio de Estados (Q):** Conjunto de subproblemas discretos. Es la *memoria mínima y suficiente* para tomar decisiones futuras sin violar el principio de optimalidad de Bellman.
- **Alfabeto de Decisiones (Σ):** Conjunto finito de transiciones o decisiones atómicas permitidas.
- **Función de Transición ($\delta : Q \times \Sigma \rightarrow Q \cup \{\emptyset\}$):** Regla determinista que calcula el sucesor $q' = \delta(q, a)$. Si la decisión a infringe restricciones, conmuta a $\emptyset$ (transición prohibida).
- **Estado Inicial ($q_0 \in Q$):** Caso base trivial de partida.
- **Conjunto de Aceptación ($F \subseteq Q$):** Estados finales que representan la resolución del problema global.

4.2 Las 3 Preguntas Universales de Diseño

Ante cualquier DP complejo en competencia, responde en papel:

1. **Eje de Avance:** ¿Qué variable escalar cuantifica el progreso monótono hacia F ? (índice, suma, tiempo, bits encendidos). Esto garantiza ausencia de ciclos y define el orden topológico.
2. **Memoria Mínima:** ¿Cuál es la mínima información histórica indispensable para dictaminar si $\delta(q, a)$ es legal? (purgar toda variable que no condicione el futuro).
3. **Decisiones Legales:** ¿Qué aristas Σ emanan del estado actual y qué invariantes o penalidades imponen?

4.3 Permutaciones vs Combinaciones (1D vs 2D)

Al contar composiciones de un alfabeto $\Sigma = \{c_1, \dots, c_n\}$ que alcanzan una suma S :

- **Permutaciones (1D - Dice / Coin Comb. I):** El estado es solo la suma $s \in Q$. En cada paso se itera sobre *todo el alfabeto* Σ :

$$dp[s] = \sum_{c \in \Sigma} dp[s - c]$$

Distingue órdenes distintos: $[1, 2]$ y $[2, 1]$ forman caminos distintos en el DAG.

- **Combinaciones (2D - Knapsack / Coin Comb. II):** Para colapsar órdenes equivalentes, se expande el estado a (i, s) acotando el prefijo de decisiones permitidas a $\{c_1, \dots, c_i\}$:

$$dp[i][s] = dp[i-1][s] + dp[i][s - c_i]$$

Fuerza un orden topológico no decreciente, garantizando que cada combinación se cuente una sola vez.

4.4 Isomorfismo Universal con Grafos (DAGs)

Toda recurrencia de DP se reduce a caminos sobre el DAG inducido $G = (V, E)$ donde $V = Q$ y $E = \{(u, v) \mid \exists a : \delta(u, a) = v\}$:

- **Mínimo Costo:** Camino más corto desde q_0 hasta F .
- **Máximo Beneficio:** Camino más largo en el DAG.
- **Conteo de Maneras:** Número de caminos dirigidos entre q_0 y F (convergencia modular).

5. Grillas ($R \times C$) y Extrapolaciones

5.1 Mapeo Lineal (Flattening)

Convierte coordenadas 2D a un ID entero 1D:

```
auto get_id = [&](int r, int c) { return r * C + c; };
auto get_row = [&](int id) { return id / C; };
auto get_col = [&](int id) { return id % C; };
```

5.2 Bipartición de Grillas (Tablero de Ajedrez)

Toda grilla con movimientos ortogonales (arriba, abajo, izquierda, derecha) es un **grafo bipartito**:

- Conjunto A (Blancas): $(r + c) \pmod{2} == 0$.
- Conjunto B (Negras): $(r + c) \pmod{2} == 1$.

Aplicaciones Inmediatas:

- *Colocar dominós 1×2 sin superposición:* Es exactamente el Emparejamiento Bipartito Máximo (Max Bipartite Matching) entre Blancas y Negras.
- *Mínimas celdas a bloquear para evitar dominós:* Por el Teorema de König, en grafos bipartitos:

$$\text{Min Vertex Cover} = \text{Max Matching}$$

$$\text{Max Independent Set} = (\text{Total Celdas}) - \text{Max Matching}$$

5.3 Flujo con Capacidad en Celdas

Si una celda (r, c) solo puede ser atravesada K veces por caminos disjuntos:

- Divide cada celda u en dos subnodos: u_{in} y u_{out} .
- Agrega arista interna: $u_{in} \rightarrow u_{out}$ con capacidad K .
- Para vecinos transitables v , conecta: $u_{out} \rightarrow v_{in}$ con capacidad ∞ .

6. Reconocimiento de Problemas Complejos

6.1 Min-Cut / Project Selection (Selección de Proyectos)

Patrón del Enunciado:

- Hay proyectos que otorgan una ganancia positiva P_i .
- Hay herramientas o licencias que cuestan C_j .
- Restricción de dependencia: "Para realizar el proyecto i , es obligatorio comprar la herramienta j ".
- Se desea maximizar la ganancia neta.

Reducción a Min-Cut:

1. Conecta la Fuente $S \rightarrow \text{Proyecto}_i$ con capacidad P_i .
2. Conecta cada Herramienta $j \rightarrow \text{Sumidero } T$ con capacidad C_j .
3. Conecta $\text{Proyecto}_i \rightarrow \text{Herramienta}_j$ con capacidad ∞ .
4. Respuesta = $\sum(\text{Todas las Ganancias}) - \text{Dinic.min_cut}(S, T)$.

6.2 2-SAT Implícito

Patrón del Enunciado: Problemas donde cada elemento tiene exactamente 2 estados excluyentes (0 o 1, Izquierda o Derecha, Sala A o Sala B) y te dan restricciones por pares:

- "No pueden estar ambos activos": $\neg(A \wedge B) \iff (A \implies \neg B) \wedge (B \implies \neg A)$.
- "Al menos uno debe estar activo": $(A \vee B) \iff (\neg A \implies B) \wedge (\neg B \implies A)$.
- "Deben ser iguales": $(A \iff B) \iff (A \implies B) \wedge (B \implies A) \wedge (\neg A \implies \neg B) \wedge (\neg B \implies \neg A)$.

Solución: Si x y $\neg x$ están en la misma componente fuertemente conexa (SCC), es imposible. De lo contrario, la asignación es válida.

6.3 Flujos con Demandas Mínimas (Circulación)

Si cada arista $e = (u, v)$ exige un flujo dentro de un rango $[L_e, R_e]$:

1. Reemplaza la capacidad por $R_e - L_e$.
2. Crea una super-fuente S' y un super-sumidero T' .
3. Para cada vértice u , define su balance de demanda:

$$D[u] = \sum_{e \text{ entra a } u} L_e - \sum_{e \text{ sale de } u} L_e$$

4. Si $D[u] > 0$, añade arista $S' \rightarrow u$ con capacidad $D[u]$.
5. Si $D[u] < 0$, añade arista $u \rightarrow T'$ con capacidad $-D[u]$.
6. Si el flujo máximo desde S' satura todas las aristas que salen de S' , existe una circulación válida.

6.4 Teorema de Dilworth (Anticadenas en DAGs)

En un conjunto parcialmente ordenado (DAG transitivo):

- **Cadena:** Secuencia donde cada elemento es menor que el siguiente.
- **Anticadena:** Conjunto de elementos donde ninguno es comparable con otro.
- **Teorema:** El tamaño de la **máxima anticadena** es igual al número mínimo de caminos disjuntos en vértices que cubren todo el DAG (resoluble con Bipartite Matching duplicando vértices).

6.5 BFS 0-1 (Grafos Implícitos)

Si las transiciones entre estados tienen costo estrictamente 0 o 1:

- **NUNCA** uses Dijkstra ($O(E \log V)$).
- Usa **ZeroOneBfs** con `std::deque`:
 - Si la arista pesa 0: `dq.push_front({v, d})`.
 - Si la arista pesa 1: `dq.push_back({v, d + 1})`.

Complejidad $O(V + E)$ en tiempo lineal estricto.

6.6 Meet-in-the-Middle

Señal inequívoca: $N \approx 36 \dots 42$.

- $2^{40} \approx 10^{12} \implies$ Imposible en 1 segundo.
- Divide en dos mitades de tamaño $N/2 \approx 20$:
 - Mitad 1: Genera los $2^{20} \approx 10^6$ valores en un vector A y ordénalo.
 - Mitad 2: Para cada uno de los 2^{20} valores x de la mitad 2, busca en A con `std::lower_bound` el complemento óptimo.

Complejidad: $O(2^{N/2} \cdot \frac{N}{2}) \approx 2 \times 10^7$ operaciones (entra sobrado en 1 segundo).

6.7 Búsqueda Binaria sobre la Respuesta

Sospecha inmediata ante las siguientes frases:

- "Minimizar el máximo valor..."
- "Maximizar el mínimo valor..."
- "Halla el menor tiempo tal que..."

Verifica la propiedad de monotonía: Si con valor M es posible, ¿también es posible con $M + 1$? Si la respuesta es sí, busca binariamente en el rango `[low, high]`.

7. Strings y Hashing Anti-Hack

7.1 Rolling Hash con Base y Módulo Seguros

En plataformas como Codeforces o competencias con tests generados con cadenas de Thue-Morse, el módulo $10^9 + 7$ o bases fijas como 31 son vulnerables a colisiones forzadas intencionalmente. Usa siempre:

- Módulo grande primo: $M = 10^9 + 9$ o $2^{61} - 1$.
- Base aleatoria generada en tiempo de ejecución: $B \in [257, M - 2]$ con `rng`.

```
const ll MOD = 1e9 + 9;
ll B; // Inicializado en main con rand_val(300, MOD - 2)

struct StringHash {
    vll h, p;
    StringHash(const string& s) {
        int n = s.size();
        h.assign(n + 1, 0); p.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h[i + 1] = (h[i] * B + s[i]) % MOD;
            p[i + 1] = (p[i] * B) % MOD;
        }
    }
    // Hash del substring s[l..r] (0-based)
    ll get(int l, int r) {
        ll res = (h[r + 1] - h[l] * p[r - l + 1]) % MOD;
        return (res + MOD) % MOD;
    }
};
```

7.2 Cuándo usar KMP vs Z-Algorithm vs Trie

- **KMP (π):** Ideal para encontrar el periodo más corto de una cadena, prefijo que es sufijo, o autómata de búsqueda de 1 patrón en streaming.
- **Z-Algorithm (Z):** Ideal para saber para cada sufijo $S[i \dots]$ cuál es el prefijo común más largo (LCP) con toda la cadena. Es mucho más intuitivo de modificar que KMP.
- **Aho-Corasick:** Indispensable si hay **múltiples patrones** simultáneos que buscar en un solo texto.

8. Geometría Computacional Segura

8.1 Producto Cruz Entero (Sin Floats)

Siempre que los puntos tengan coordenadas enteras ($\leq 10^8$), calcula la orientación usando `long long` estricto:

```
ll cross(Point a, Point b, Point c) {
    return 1LL * (b.x - a.x) * (c.y - a.y) - 1LL * (b.y - a.y) * (c.x - a.x);
}
// cross > 0: Giro antihorario (a la izquierda)
// cross < 0: Giro horario (a la derecha)
// cross == 0: Puntos colineales
```


8.2 Convex Hull: Colinealidad en el Borde

En el algoritmo de Monotone Chain:

- **Cáscara estricta (sin colineales):** Usa `cross(...) <= 0`.
- **Preservando colineales en el borde:** Usa `cross(...) < 0`.

8.3 Intersección de Segmentos Entera

Para saber si dos segmentos AB y CD se cruzan estrictamente sin usar números flotantes:

```
bool intersect(Point a, Point b, Point c, Point d) {
    auto cp = [](Point p, Point q, Point r) {
        ll v = cross(p, q, r);
        return (v > 0) - (v < 0);
    };
    return cp(a, b, c) * cp(a, b, d) < 0 &&
           cp(c, d, a) * cp(c, d, b) < 0;
}
```

8.4 Calipers Rotativos (Rotating Calipers)

Calcula el diámetro de un polígono convexo en tiempo $O(N)$ encontrando los pares de puntos antípodas mediante avance de puntero monótono en sentido antihorario.

9. Checklist de Emergencia Pre-Submit

Antes de presionar Submit y arriesgar una penalización de 20 minutos por Wrong Answer:

- [] **¿Hay desbordamiento entero (Integer Overflow)?** ¿Alguna multiplicación de dos enteros de 10^5 o suma acumulada supera 2×10^9 ? Usa `1LL * a * b` o define `#define int long long`.
- [] **¿El valor de INF es suficientemente grande?** En Dijkstra con pesos de arista hasta 10^9 , `INT_MAX` se desborda inmediatamente. Usa `LINF = 2e18`.
- [] **¿Múltiples casos de prueba (T testcases)?** Verifica que las estructuras globales se limpien con precisión hasta N , no reasignando vectores gigantes con `memset` innecesario que cause TLE ($O(T \cdot \text{MAXN})$).
- [] **Casos de Borde (Corner Cases):**
 - $N = 1$ o $N = 2$.
 - Árbol con forma de estrella o línea recta (máxima profundidad).
 - Grafo desconexo (¿el problema exige imprimir -1 ?).
 - Cadenas vacías o caracteres todos idénticos.
 - Polígonos de área 0 o puntos superpuestos.

9.1 Script de Stress Testing Local (3 minutos)

Cuando tengas un código con WA y un algoritmo de fuerza bruta confiable ($O(N^2)$), crea un script de testeo rápido en Python:

```
import random, subprocess

for it in range(1000):
    n = random.randint(1, 10)
    a = [random.randint(1, 20) for _ in range(n)]
    data = f"{n}\n" + " ".join(map(str, a)) + "\n"

    out1 = subprocess.check_output("./sol", input=data, text=True)
    out2 = subprocess.check_output("./brute", input=data, text=True)

    if out1.strip() != out2.strip():
        print(f"FAILED on test:\n{data}")
        print(f"Sol: {out1}\nBrute: {out2}")
        break
```

10. Árbol de Navegación del Cuadernillo (Plantilla PDF)

Mapa jerárquico de contenidos y número de página exacto en `plantilla.pdf` para búsqueda ultrarrápida durante la competencia:

```
[Pag. 1] MAIN & CONFIG (src/main.cpp)
+-- Fast I/O, Pragmas, Typedefs (ll, vi, vll, pii)
+-- Constantes (INF, LINF, MOD, EPS)

[Pags. 1-8] GRAPHS (include/graph.hpp)
+-- [P. 1] Direcciones grilla (dx4, dy4, dx8, dy8), isValid
+-- [P. 1] DSU (Union-Find con component_size y members)
+-- [P. 1] Graph<T> & Edge<T> (1-based)
+-- [P. 1-2] Dijkstra<T> (distancias y restore_path)
+-- [P. 2] ZeroOneBfs<T> (0-1 BFS con deque)
+-- [P. 2] BFS / DFS clasico & ciclo en dirigido
+-- [P. 2-3] Bellman-Ford & SPFA (ciclos negativos)
+-- [P. 3] Floyd-Warshall (all-pairs shortest path)
+-- [P. 3] Kruskal (MST con DSU) & Prim
+-- [P. 3-4] Topological Sort (Kahn con queue)
+-- [P. 4] Kosaraju / Tarjan (SCC componentes conexas)
+-- [P. 4-5] 2-SAT (TwoSat con asignacion booleana)
+-- [P. 5] Bridges & Articulation Points (Puntos de corte)
+-- [P. 5-6] Euler Path / Circuit (Hierholzer)
+-- [P. 6-7] Dinic (Max Flow con BFS niveles y DFS)
+-- [P. 7-8] Min Cost Max Flow (MCMF con potenciales)
+-- [P. 8] Dominator Tree & Bipartite Matching (Hopcroft)

[Pags. 9-14] TREES (include/tree.hpp)
+-- [P. 9] Diametro y Centro de un arbol
+-- [P. 9-10] Binary Lifting & LCA (k-esimo ancestro)
+-- [P. 10-11] Heavy-Light Decomposition (HLD)
+-- [P. 11-12] Centroid Decomposition (solve por caminos)
+-- [P. 12] Euler Tour / Flattened Tree (tin/tout)
+-- [P. 13] DSU on Tree (Sack / Small to Large  $O(N \log N)$ )
+-- [P. 14] Virtual Tree / Arbol Auxiliar

[Pags. 15-20] RANGE QUERIES (include/rq.hpp)
+-- [P. 15] Sparse Table (0-based, RMQ  $O(1)$ )
+-- [P. 15] Fenwick Tree / BIT (1-based, sumas y rangos)
+-- [P. 15-16] 2D BIT & Range Update BIT
+-- [P. 16] Iterative Segment Tree (0-based, point update)
+-- [P. 16-17] Recursive SegTree con Lazy Propagation
+-- [P. 18] Dynamic / Implicit SegTree (nodos punteros)
```

```

+-- [P. 18-19] Merge Sort Tree (conteo  $\leq k$  en rango)
+-- [P. 19-20] Persistent Segment Tree (versiones
    historicas)
+-- [P. 20] Mo's Algorithm con bloques sqrt y Hilbert

[Pags. 21-24] STRINGS (include/string.hpp)
+-- [P. 21] KMP / Prefix Function (pi array)
+-- [P. 21] Z-Algorithm (Z-array  $O(N)$ )
+-- [P. 21-22] StringHash (Rolling Hash doble modulo)
+-- [P. 22] Trie estandar (arbol de prefijos)
+-- [P. 22-23] Aho-Corasick (automata multi-patron)
+-- [P. 23] Suffix Automaton (SAM lineal  $O(N)$ )
+-- [P. 23-24] Suffix Array & LCP Array (Kasai)
+-- [P. 24] Manacher (palindromos en  $O(N)$ )

[Pags. 25-26] DYNAMIC PROGRAMMING (include/dp.hpp)
+-- [P. 25] LIS (Longest Increasing Subsequence  $O(N \log N)$ )
+-- [P. 25] Knapsack 0/1, Infinito y Acotado
+-- [P. 25] Matrix Exponentiation para DP lineal
+-- [P. 25-26] Convex Hull Trick (LineContainer dinamico
    )
+-- [P. 26] Divide and Conquer DP Optimization
+-- [P. 26] SOS DP (Sum Over Subsets en  $O(N 2^N)$ )
+-- [P. 26] Tree Rerooting DP (despacho de raiz  $O(N)$ )

[Pags. 27-31] MATHEMATICS (include/math.hpp)

```

```

+-- [P. 27] Criba lineal & Factorizacion rapida
+-- [P. 27] Binpow, Inverso modular, Combinatoria nCr
+-- [P. 28] ExtGCD & Teorema Chino del Resto (CRT)
+-- [P. 28] Miller-Rabin & Pollard-Rho  $O(N^{0.25})$ 
+-- [P. 29-30] NTT / FFT (Convolucion polinomial  $O(N \log N)$ )
+-- [P. 30-31] Gauss-Jordan & Base XOR (Linear Basis)

[Pags. 32-34] GEOMETRY (include/geometry.hpp)
+-- [P. 32] Point struct, dot, cross, norm, orient
+-- [P. 32] Interseccion y distancias a lineas/segmentos
+-- [P. 32-33] Convex Hull (Monotone Chain con
    colineales)
+-- [P. 33] Rotating Calipers (diametro maximo)
+-- [P. 33] Area de poligono y punto en poligono
+-- [P. 33-34] Interseccion de Semiplanos (Halfplane)
+-- [P. 34] Circulos (interseccion con linea y circulo)

[Pag. 35] BIT OPERATIONS (include/bits.hpp)
+-- Builtins (__builtin_popcount, clz, ctz)
+-- Iteracion de submascaras sub = (sub - 1) & mask

[Pags. 36-37] EXTRA (include/extra.hpp)
+-- Custom Hash para unordered_map / set
+-- Compresion de coordenadas (coordinate compression)
+-- Fast I/O & Custom Allocator

```